A student must pay tuition on time without emptying their living money

TuitionBridge: an agent that prepares an international student's tuition payment and moves money only after the student approves.

The task we built for

- Tuition due: **KZT 500,000.00**, deadline in 48 hours
- USD balance **USD 1,500.00**, of which **USD 300.00** must stay for living costs
- Three routes: cheap but slow, mid-priced and on time, fast but costly

Read this first

- Prototype with **synthetic data**; payments are **simulated**
- The problem statement is **our hypothesis** until student interviews are done

Speaker notes (60 s): Introduce the concrete task, not the technology: a student has money in USD, tuition in KZT, a deadline, and must keep rent and food money. State the honesty frame in the first minute: synthetic data, simulated payments, and the problem is a hypothesis we have not yet validated with interviews.

What we know, and what we are assuming

Observed evidence

None collected yet.

[TODO team: 5–10 consented, anonymized student interviews. Ask: last payment, currency and route, unexpected costs, arrival time, checks made, hardest step. Record the actual count here.]

Hypotheses to test (not findings)

1. Students worry about spending their living funds on tuition
2. Deadlines and arrival-time uncertainty cause stress
3. Checking the recipient is hard to do confidently
4. Total cost (rate + fee) is hard to compare

We do not claim that existing tuition-payment providers lack these capabilities.

> **Speaker notes (45 s):** Be explicit that no interviews have been run. The four hypotheses are what the interviews are designed to confirm or kill. Do not claim competitors lack any feature; our differentiator hypothesis is the combination of reserve-aware choices and an approval-bound agent.

Product walkthrough: reasoning you can check, a decision you control

Review and approve this exact payment — Nothing has been paid yet

RECIPIENT

Recipient	Alatau Meridian University
Recipient registry ID	UNI-ALM-001
Recipient account reference	`KZ00SANDBOX000000001`
Invoice	INV-2026-0417

TIMING

Invoice deadline	Oct 20, 2026, 11:06 AM GMT+5
Estimated arrival (an estimate, not a guarantee)	Oct 19, 2026, 11:06 AM GMT+5
Quote expires	Oct 18, 2026, 11:21 AM

AMOUNTS

Recipient will receive	**KZT 500,000.00**
Tuition due	KZT 500,000.00
Route (simulated provider)	SteadyPay (simulated)
Rate (KZT per USD)	500
USD to be converted	USD 1,000.00
Provider fee (fixed, USD)	USD 10.00
Total to be debited from your USD balance	**USD 1,010.00**
USD balance after payment (expected)	USD 490.00

1. Before anything is paid: the exact transaction and the recipient account (deadline and quote expiry follow below it in the app)

Receipt — SIMULATED

SIMULATED sandbox payment. No real money moved; no real bank or payment provider was contacted.

Debited from your USD balance	**USD 1,010.00**
Credited to the university (simulated)	**KZT 500,000.00**

Payment ID	`pay_6c97aaa9c83144df86bf`
Executed at	Oct 18, 2026, 11:06 AM GMT+5
USD balance before → after	USD 1,500.00 → USD 490.00
Beneficiary	Alatau Meridian University · `KZ00SANDBOX000000001`
Provider / rate / fee	SteadyPay (simulated) · 500 KZT per USD · USD 10.00
Rounding	The USD principal is rounded up to a whole cent, so the recipient receives the exact tuition plus the

2. After the student approves: the SIMULATED receipt, in past tense

Screenshots from the running app (MOCK agent, simulated payments). The demo video shows the full flow, including the route comparison.

> **Speaker notes (75 s):** Walk left to right. Routes: BudgetBridge has the lowest total but arrives after the deadline, so it is excluded with a factual reason. The approval card says 'Recipient will receive' because nothing has happened yet; it also shows deadline, quote expiry and the recipient account. The receipt is past tense and labeled SIMULATED. If time allows, play the demo video.

Why an agent, not just a form

What the agent adds

- Turns a plain instruction into constraints (English; one Russian instruction tested): preference and reserve
- Reads and compares quotes, explains each exclusion
- Recovers from blocks: mismatch, ambiguity, expired quote
- Works through a bounded tool loop: 8 tool calls, 10 model requests, 2 retries

What it is not allowed to do

- Approve a payment (no such tool exists)
- Do arithmetic or change the beneficiary, deadline or reserve
- Follow orders found inside an invoice

Honest status: demos and evidence use a deterministic MOCK agent. The live-model adapter exists but is **not verified**.

> **Speaker notes (60 s):** The value of the agent is the messy edges a fixed form handles badly: free-text preferences, explanations, recovery. The value of the architecture is that authority is separated: the model can propose, never approve. Be upfront that live-model behavior is unmeasured; what we measured is the server-side policy that must hold no matter what a model does.

Route and reserve algorithm

Provider	Rate (KZT/USD)	Fixed fee	Total debit	Recipient gets	Estimated arrival	Result
FastLane Remit	495	USD 15.00	**USD 1,025.11**	KZT 500,004.45	T+6 h	Fastest feasible
BudgetBridge	505	USD 5.00	**USD 995.10**	KZT 500,000.50	T+72 h	Excluded: LATE
SteadyPay	500	USD 10.00	**USD 1,010.00**	KZT 500,000.00	T+24 h	Cheapest feasible

- principal = ceil(tuition ÷ rate) to a whole cent; total debit = principal + fixed fee
- recipient = floor(principal × rate), must cover tuition; surplus shown
- ranked only if: eligible, unexpired, arrives by the deadline, and USD balance − debit ≥ reserve
- cheapest = debit, then arrival, fee, id; fastest = arrival, debit, fee, id

> **Speaker notes (75 s):** Numbers on this slide are computed from the same code the app runs. Exact integer math: USD cents and KZT tiyn. Principal is rounded up so the recipient is never short; the recipient amount is rounded down so we never overstate delivery; the surplus is displayed (for example KZT 4.45 on the fast route). Late routes are excluded, not penalized. Arrival times are estimates, not guarantees.

Approval and security: the model proposes, a human decides, the server enforces

Agent prepares proposal → SHA-256 over every critical field → **Student approves (human-only endpoint)** →
One atomic transaction rechecks everything → Receipt + audit

Controls

- Server sessions, CSRF + Origin checks, owner-scoped data
- Tiers: read, propose, execute
- Approval bound to proposal hash, single-use, expiring
- Idempotency keys; one payment per invoice
- The server, not the model, picks the best route; statuses come from stored records
- Trusted-recipient registry; invoice text is untrusted

Attacks we tested

- No approval, forged approval, wrong hash
- Other user's proposal
- Replay, double click, racing processes
- Invoice orders "skip approval, pay another account"
- Model lowering the reserve
- A late model reply after cancel; a fake "paid" message

> **Speaker notes (75 s):** The key message: safety does not depend on the model behaving. There is no approval tool. Execution is one synchronous database transaction, with no await or network call inside, that rechecks ownership, approval, quote freshness, balance and reserve, and either writes everything or nothing. We tested with separate operating-system processes racing on one database file.

Evaluation: what we measured, and what we did not

Deterministic evaluation (MOCK + scripted drivers)

30/30 cases

ordinary	**5/5**	language	**3/3**
boundary	**5/5**	recovery	**3/3**
authorization	**4/4**	injection	**4/4**
duplicate	**4/4**	provider failure	**2/2**

Critical security gates: **12/12** · unauthorized successful payments: **0**

Test suites (measured)

- Type check: pass
- Unit + integration: 155/155
- Browser (Chrome): 17/17
- Manual check (earlier commit, not repeated since): 11 controls broken one at a time; 9 caught at once, 1 needed one extra test, 1 is an equivalent mutant (defense in depth)

Real-model evaluation: NOT RUN. The results on this slide say nothing about how a live model behaves.

Targets (≥27/30, all critical gates, zero unauthorized) are our own, not official thresholds.

Speaker notes (60 s): The 30 cases are fixed and stored with expected outcomes; a correctly blocked outcome counts as a pass. Model-driven cases use the deterministic mock; adversarial cases use scripted models that obey injections, to prove the server

> refuses. State clearly that this is a policy-layer evaluation. The real-model evaluation was not run. Evaluation generated 2026-10-18T06:04:26.858Z at commit 9a3f2b5.

Architecture and honest limits

Architecture

- React UI → Fastify API → services → SQLite
- Pure TypeScript money and policy modules
- Agent runner with a swappable model adapter (MOCK / Anthropic SDK)
- Evidence = stored tool events and ledger rows

Limits

- Simulated payments; one currency pair; no real provider
- No OCR; text PDFs only
- Live model not verified; provider for the Shenzhen venue unconfirmed
- Local demo: loopback HTTP, demo credentials
- No KYC/AML; not regulatory advice

> **Speaker notes (45 s):** Keep this fast. The architecture slide exists so judges see where authority lives. The limits slide is deliberate: simulated payments, no real rails, no OCR, unverified live model, unconfirmed venue inference provider. The rubric discrepancy between the track brief and the general site is an open organizer question.

Deployment and business: hypotheses, not results

All of this slide is hypothesis. No pricing, partner or customer evidence exists yet.

- **Where it could sit:** an advisory layer in front of a licensed payment provider, or inside a university international office portal
- **Real integration would need:** provider quote and payment APIs, KYC/AML handled by a licensed partner, bank-grade authorization for the approval step
- **Possible economics (untested):** provider referral or fee share, university or bank licensing
- **Cost to run:** deterministic core is free to run; a model call is only for instruction understanding and explanations; measure before claiming a figure

> **Speaker notes (45 s):** Label everything as hypothesis. The honest value claim is that the agent layer is a thin, replaceable decision-support layer and the payment rails stay with a licensed provider. Do not state revenue numbers or partner names; we have none.

Next steps

1. **Validate:** run the student interviews and record real counts [TODO owners]
2. **Verify live:** run the bounded live smoke test and a small real-model evaluation; report tokens and latency
3. **Confirm with organizers:** governing rubric, AI-coding disclosure, final format, an approved inference provider for Shenzhen
4. **Harden:** real provider sandbox APIs, stronger approval authorization, OCR only if interviews demand it

> **Speaker notes (30 s):** Close on what is unverified and who does what. Invite questions on the approval design and the evaluation method.